

CE 791 HPC - Assignment 4: Simple MPI

Andrew Navratil

Part 1. Pi Example

The pi example program in Fortran was modified to include a timer with `MPI_Wtime` and an outer loop to perform 100 trials of 1 million intervals to get a better sense of the Mflops rating. The number of flops per interval was estimated as 4: 1 for the x assignment multiplication and subtraction, one for the sum assignment addition, and two inside the function call `f(a)` with one for the division and one for the multiplication and addition. Code was executed with `mpirun` on henry2 for 1,2,4, and 8 processors. Resulting values for Mflops and Time were output to the console and then copied into Matlab code for plotting.

We can see in the plots below that time decreases as number of processors increases - they are inversely proportional. The Mflops increases linearly with the number of processors, as expected for this embarrassingly parallel problem.

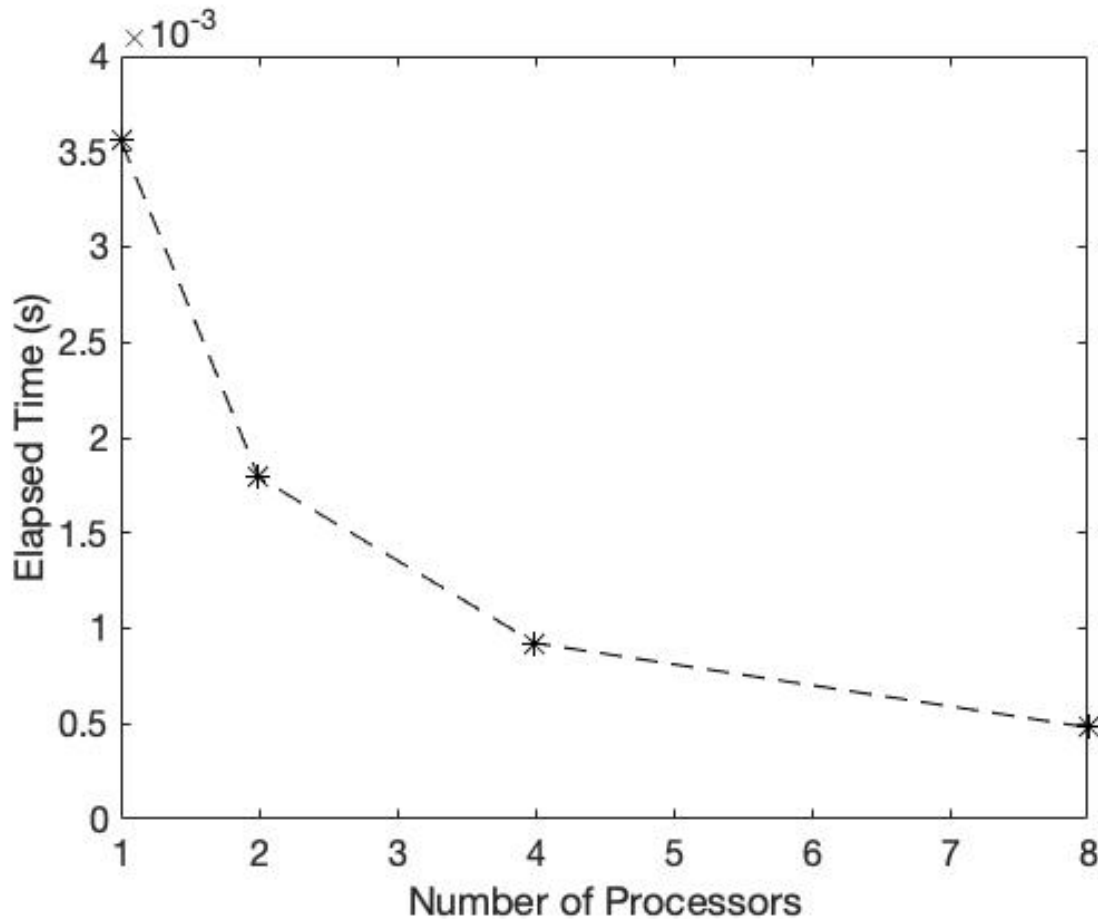


Fig. 1 Execution Time for fpi on henry2

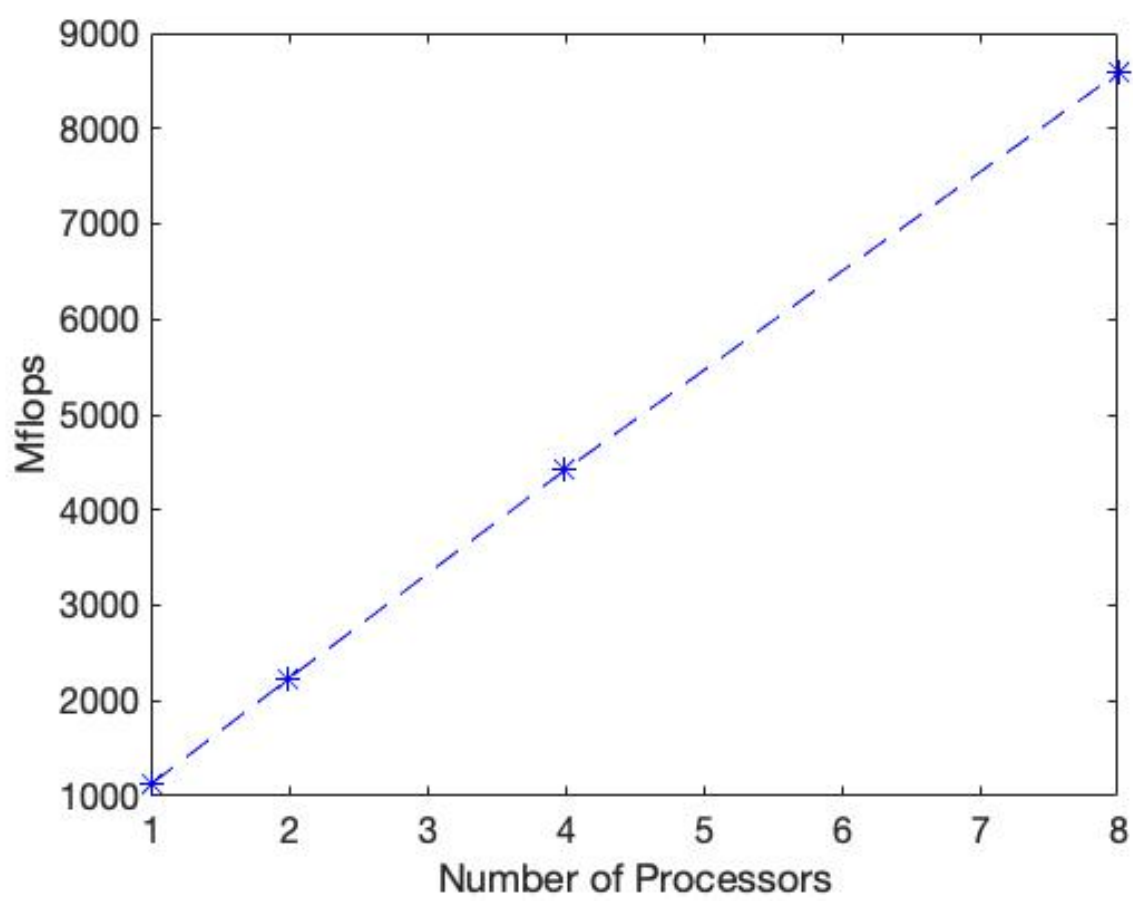


Fig. 2 Mflops for fpi on henry2

Part 2. Water Supply

The water supply vector program in Fortran was modified to include a timer with `MPI_Wtime`, an outer loop to perform 10 trials of 1 million intervals, and `MPI_Reduce` with `MPI_MIN`. Resulting values of Mflops were a bit less than the non-parallelized code, more investigation needst to be done into the best way to parallelize all three loops in this code. Investigation started into `MPI_Gather` and could continue there. Code was executed with `mpirun` on `henry2` for 1,2,4, and 8 processors. Resulting values for Mflops and Time were output to the console and then copied into Matlab code for plotting.

We can see in the plots below that both time and Mflops degrade as number of processors is increased, showing the need to write a better parallel algorithm for this part of code.

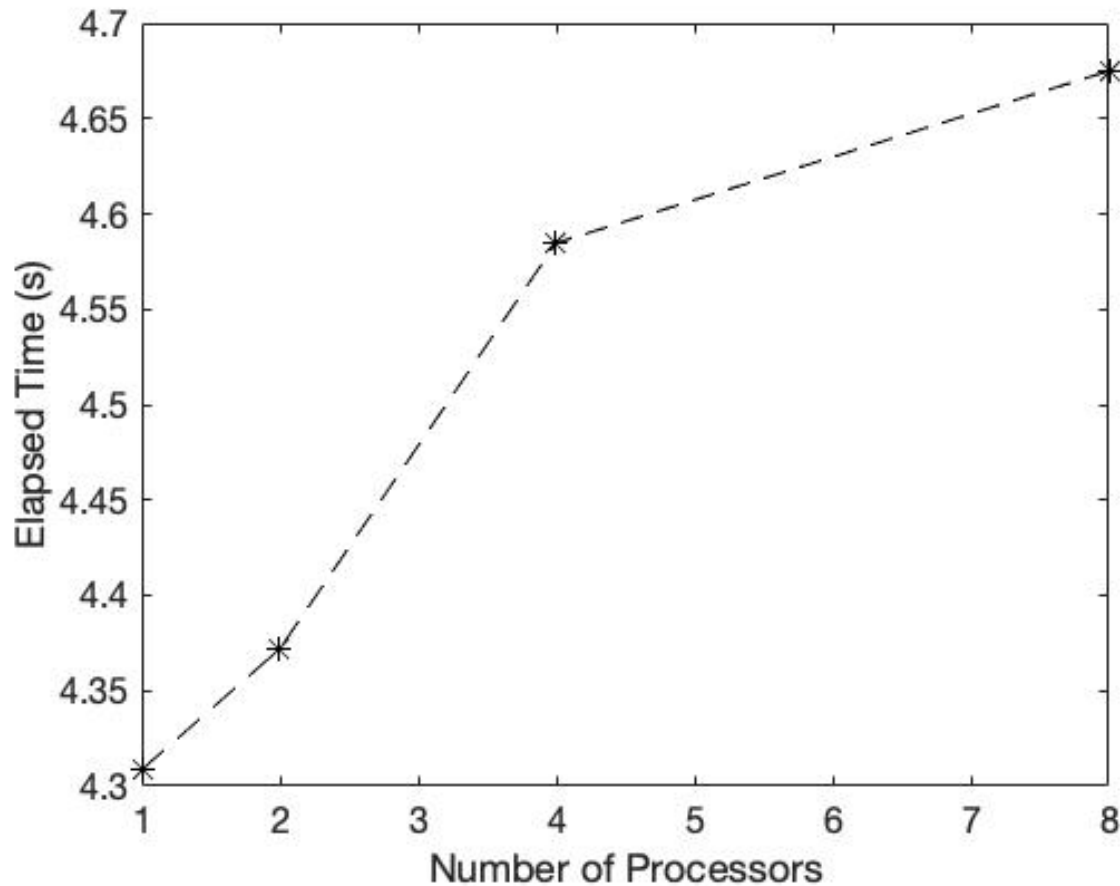


Fig. 3 Execution Time for water_supplyf on henry2

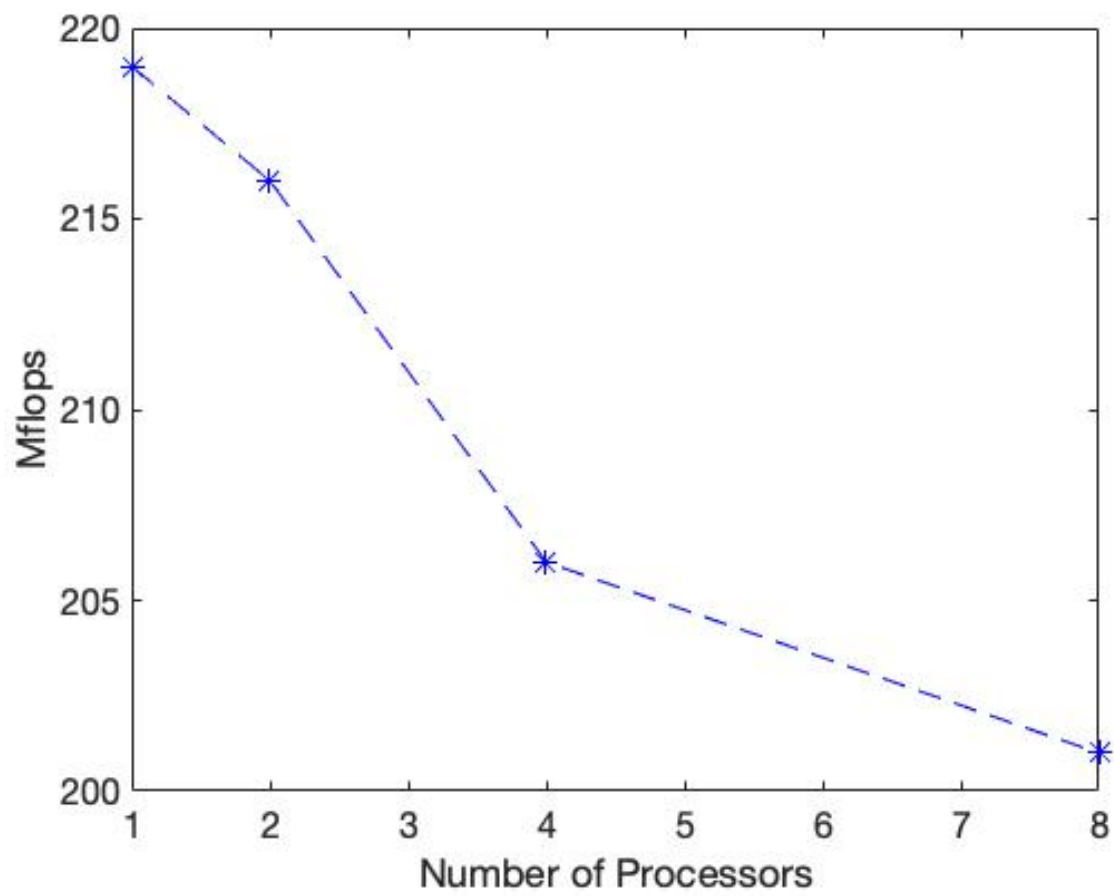


Fig. 4 Mflops for water_supplyf on henry2

Fortran Code: Pi

```
1  program main
2  ! use mpi
3  ! Use the following include if the mpi module is not available
4  include "mpif.h"
5  real(8), parameter :: PI25DT = 3.141592653589793238462643d0
6  real(8) mypi, pi, h, sum, x, f, a, time1, time2, total_time
7  integer n, myid, numprocs, i, ierr, ntimes, j
8  ! function to integrate
9  ! this is a statement function (obsolete – move to an internal function)
10 f(a) = 4.d0 / (1.d0 + a*a)
11
12 call MPI_INIT(ierr)
13 call MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)
14 call MPI_COMM_SIZE(MPI_COMM_WORLD, numprocs, ierr)
15
16 ! start timer
17 time1=MPI_Wtime()
18
19 ! calculate the interval size
20 n = 1000000
21 h = 1.0d0/n
22
23 ! outer loop to perform multiple trials
24 ! ntimes=1024 ! too big to fit in int when multiplied by 3*n*numprocs
25 ntimes=100
26 do j = 1, ntimes
27
28     sum = 0.0d0
29     do i = myid+1, n, numprocs
30         x = h * (dble(i) - 0.5d0)
31         sum = sum + f(x)
32     enddo
33     mypi = h * sum
34 ! collect all the partial sums
35 call MPI_REDUCE(mypi, pi, 1, MPI_DOUBLE_PRECISION, MPI_SUM, 0, &
36                MPI_COMM_WORLD, ierr)
37
38 ! end outer num trials loop
39 enddo
40
41 ! end timer
42 time2=MPI_Wtime()
43
44 ! calculate avg time per trial: time2-time1 gives total time
45 total_time=(time2-time1)/ntimes
46
47 ! node 0 prints the answer.
48 if (myid.eq.0) then
49     print *, 'pi is ', pi, ' error is', abs(pi - PI25DT)
50
51     ! 4 flops per loop (x has *-, sum has +, f(a) has / and *+)
```

```
52     mflops = 4*n*1e-6/(total_time)
53
54     !print *, n, ' ', ntimes, ' ', numprocs, ' ', 3*n*ntimes*numprocs
55     print *, 'mflops =', mflops
56     print *, 'total time =', total_time
57
58     endif
59     call MPI_FINALIZE(ierr)
60     end
```

Fortran Code: Water Supply

```
1  ! program to perform enumeration for a water supply problem
2  Program water_supply
3      use ifport
4      !use mpi
5      include "mpif.h"
6  !declare variables
7      integer, parameter :: ntrials = 10000000, iseed=567891
8      real(8) x1(ntrials), x2(ntrials), x3(ntrials), cost(ntrials)
9      real(8) sl1, sl2, sl3, hc1, hc2, hc3, demand, uc1, uc2, uc3, hardness
10     real(8) time1, time2, time3, time4
11     real(8) min_cost, time_obj, time_gen, elapsed_time, mflops
12     integer min_cost_index
13
14     integer n, myid, numprocs, ierr, ntimes
15     integer i,j,k
16     real(8) min_cost_global
17
18     sl1=25; sl2=120; sl3=100; hc1=200; hc2=2300; hc3=700
19     demand=150; uc1=500; uc2=1000; uc3=2000
20
21     call MPI_INIT(ierr)
22     call MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)
23     call MPI_COMM_SIZE(MPI_COMM_WORLD, numprocs, ierr)
24
25     ! start timer – want to run for entire computation
26     time1=MPI_Wtime()
27
28  ! generate trial values
29     !call wtime(time1)
30
31     ! outer loop to perform multiple trials
32     !ntimes=1024 ! too big to fit in int when multiplied by 3*n*nprocs
33     ntimes=10
34     do k = 1,ntimes
35  !=====
36         j = 0
37         call seed(iseed)
38     do i=1,ntrials
39         !do i=myid+1,ntrials,numprocs
40             x3(i) = -1
41             do while (x3(i) > sl3 .or. x3(i) < 0 .or. &
42                 hardness > 1200)
43                 x1(i)=rand()*sl1
44                 x2(i)=rand()*sl2
45                 x3(i)=demand-x1(i)-x2(i)
46                 hardness=(hc1*x1(i)+hc2*x2(i)+hc3*x3(i))/demand
47             enddo
48             j=j+1
49         enddo
50     enddo
51     ! collect all the partial sums
```

```

52 ! look into this later, MPI_Gather maybe
53     !call MPI_REDUCE(x1(i)uu,pi,1,MPI_DOUBLE_PRECISION,MPI_SUM,0, &
54                     !MPI_COMM_WORLD,ierr)
55     !call wtime(time2)
56     !time_gen=time2-time1
57 !calculate cost
58     do i=1,ntrials
59         cost(i)=uc1*x1(i)+uc2*x2(i)+uc3*x3(i)
60     enddo
61     !call wtime(time3)
62     !time_obj=time3-time2
63     min_cost=1e10
64     do i=myid+1,ntrials,numprocs
65         if (cost(i) < min_cost) then
66             min_cost=cost(i)
67             min_cost_index=i
68         endif
69     enddo
70
71 ! use MPI_MIN on min_cost
72 call MPI_REDUCE(min_cost,min_cost_global,1,MPI_DOUBLE_PRECISION,MPI_MIN,0,MPI_COMM_WORLD,←
    ierr)
73
74     ! end outer num trials loop
75 enddo
76
77     ! end timer
78     time2=MPI_Wtime()
79
80     ! calculate avg time per trial: time2-time1 gives total time
81     elapsed_time=(time2-time1)/ntimes
82
83     !call wtime(time4)
84     !elapsed_time=time4-time1
85
86 !     node 0 prints the answer.
87     if (myid.eq.0) then
88         !mflops_obj=5*ntrials*1e-6/time_obj
89         mflops = dble(10*j+ntrials*5)*1e-6/elapsed_time
90
91     !print *, 'amount drawn from source1 =',x1(min_cost_index)
92     !print *, 'amount drawn from source2 =',x2(min_cost_index)
93     !print *, 'amount drawn from source3 =',x3(min_cost_index)
94     print *, 'Minimum cost = ',nint(min_cost_global), ' dollars'
95     !print *, 'Time to generate alternatives = ',time_gen, ' secs'
96     !print *, 'Time to calculate objective = ',time_obj, ' secs'
97     !print *, 'Mflop rating for Objective calculation = ',mflops_obj, ' mflops'
98     print *, 'Mflop rating for entire code = ',mflops, ' mflops'
99     print *, 'Total time = ',elapsed_time, ' secs'
100
101     endif
102     call MPI_FINALIZE(ierr)
103     end

```


Matlab Code

```
1 close all; clear all; clc;
2
3 %=====
4 % pi example
5 %=====
6 if 0
7     % fpi executed on henry2 where mflops,time,numprocs printed to console and copied ↵
8     here
9     procs = [1 2 4 8]
10    times = [3.560080528259277E-003 1.791508197784424E-003 ...
11            9.223484992980957E-004 4.815793037414551E-004]
12    mflops = [1123 2233 4421 8582]
13
14    plot(procs,times,'*k—','MarkerSize',10);
15    xlabel('Number of Processors');
16    ylabel('Elapsed Time (s)');
17    set(gca,'FontSize',16);
18
19    figure;
20    plot(procs,mflops,'*b—','MarkerSize',10);
21    xlabel('Number of Processors');
22    ylabel('Mflops');
23    set(gca,'FontSize',16);
24 %=====
25 % water supply
26 %=====
27 elseif 1
28     % water_supply_vector.f executed on henry2
29     % mflops,time,numprocs printed to console and copied here
30     procs = [1 2 4 8]
31     times = [4.30857939720154 4.37197232246399 4.58475260734558 4.67490670681000]
32     mflops = [219 216 206 201]
33
34     plot(procs,times,'*k—','MarkerSize',10);
35     xlabel('Number of Processors');
36     ylabel('Elapsed Time (s)');
37     set(gca,'FontSize',16);
38
39     figure;
40     plot(procs,mflops,'*b—','MarkerSize',10);
41     xlabel('Number of Processors');
42     ylabel('Mflops');
43     set(gca,'FontSize',16);
44 end
```